

Phase 5: System Integration & UI

Presentation Confidence Detector

Project Blueprint

Phase 5: System Integration & UI

What Is This Phase?

This is where you take four separate engines (Face, Speech-to-Text, NLP, Voice) and make them work together in one browser tab with one combined score, one dashboard, and real-time coaching alerts.

This is the hardest phase. Not because the individual pieces are complex, but because making them all run simultaneously without freezing the UI is an engineering challenge.

Think of it this way:

- Phases 1-4 = building each instrument
- Phase 5 = conducting the orchestra

Scope note: From this point onward, these docs assume a **browser-first MVP**. There are 4 processing engines, but they collapse into 3 top-level scored outputs for the user: **Face**, **Speech**, and **Voice**.

WHY This Phase Matters for the Confidence Detector

Without system integration, you have four separate demos:

- A face expression viewer
- A transcript display
- A text analyzer
- A voice pitch meter

Nobody wants four separate windows. The product is ONE dashboard that shows ONE confidence score with real-time coaching. Phase 5 is what turns a collection of demos into a product.

Skills to Learn

#	Skill	What It Is	WHY for Confidence Detector	What to Learn	Difficulty	New?
1	Multi-Engine Orchestration	Running Face, STT, NLP, and Voice engines simultaneously in one app	All 4 engines must process data in parallel — face reads camera every 66ms, STT streams continuously, NLP analyzes on each new	Web Workers, <code>setInterval</code> scheduling, <code>async/await</code> patterns, engine lifecycle (start/stop/pause)	Hard	Similar to ExamGuard (YOLO + MediaPipe ran together, but this is 4 engines not 2)

			sentence, voice analyzes audio chunks			
2	Scoring Algorithm Design	Weighted combination of the 3 scored outputs produced by the 4 engines into one 0-100 score	The user sees ONE number. Face engine produces face score, STT + NLP together produce speech score, and voice engine produces voice score. You need a formula that combines them fairly and updates smoothly	Weighted averages, normalization, smoothing (exponential moving average), handling missing signals (what if mic is off?)	Medium	Similar to ExamGuard (same weighted-average mindset, different inputs)
3	State Management	Handling real-time data from 4 engines updating at different rates in one UI	Face updates 15x/sec, STT updates on each word, NLP updates on each sentence, voice updates 4x/sec — the UI must stay in sync without lag or flicker	React state (<code>useState</code> , <code>useRef</code>), batching updates, avoiding unnecessary re-renders, shared state between components	Medium	NEW (ExamGuard's web UI was simpler — one camera, one score)
4	Real-Time Dashboard Components	Live meters, charts, waveforms, animated score displays	The dashboard IS the product. A confidence meter that animates smoothly, a live transcript with	Canvas/SVG animation, chart libraries (Chart.js or Recharts), CSS transitions, efficient DOM updates, gauge/meter components	Hard	NEW (ExamGuard had a basic table UI, not animated dashboards)

			highlighted fillers, a pitch waveform — this is what the user sees			
5	Coaching Alert System	Rule-based nudges with cooldown timers	"Slow down", "Make eye contact", "Reduce filler words" — real-time advice based on current metrics. Must not spam the user (cooldown between alerts)	Rule engine (if filler_rate > threshold, fire alert), cooldown timers, priority system (which alert is most important right now), alert queue	Medium	NEW
6	Session Recording	Storing metrics over time for the post-session report	The user wants to see how their confidence changed during the presentation — "you started strong but lost confidence at minute 3"	Time-series data storage (array of {timestamp, scores}), sampling rate decisions, memory management for long sessions	Medium	NEW
7	Report Generation	Aggregating session data into a summary with visualizations	After the session ends, show a report: overall score, timeline chart, top issues, specific moments to review	Data aggregation (averages, mins, maxs), chart rendering, section breakdown (face vs. speech vs. voice), actionable recommendations	Medium	NEW
8	AI API	Sending	"Claude,	REST API calls to	Medium	NEW

	Integration (Optional v2)	frames + transcript to Claude API for deep analysis	look at this person's face and read their transcript. What specific feedback would you give?" — adds a layer of intelligence beyond rule-based scoring	Claude API, prompt engineering for feedback, handling API latency (async), cost management (do not send every frame)		
9	Performance Optimization	Keeping 15+ FPS with multiple engines running in one browser tab	If the app drops below 10 FPS, the face mesh stutters, the transcript lags, and the experience feels broken. Performance is not optional	Profiling with Chrome DevTools, Web Worker offloading, frame skipping strategies, reducing unnecessary renders, requestAnimationFrame	Hard	Similar to ExamGuard (same challenge of real-time processing, but more engines = more load)

Skill Details

1. Multi-Engine Orchestration

What: Running all four detection engines in parallel without them blocking each other.

```

Main Thread (UI)
|
├── Face Engine ───────── Web Worker or main thread
|   └── MediaPipe Face Mesh → face score every 66ms
|
├── STT Engine ───────── Web Speech API on main thread
|   └── Microphone → transcript text (continuous)
|
├── NLP Engine ───────── Runs on each new sentence
|   └── Transcript → filler/hedge/pace analysis
|
└── Voice Engine ───────── Web Audio API + Worker
    └── Microphone → pitch/volume/tremor every 250ms

```

Confidence Detector connection: This is the backbone. If the engines do not run together smoothly, nothing else works.

What to learn:

- Starting/stopping each engine independently
- Handling engine failures gracefully (camera denied? run without face)
- Synchronizing outputs (all engines feed into one scoring function)
- Engine status indicators (green = running, red = error, gray = off)

ExamGuard comparison: ExamGuard ran YOLO + MediaPipe in a pipeline (one after another on each frame). The Confidence Detector runs 4 engines truly in parallel on different data streams. Harder, but the same mindset of "multiple AI models, one system."

2. Scoring Algorithm Design

What: Taking the 3 scored outputs produced by the 4 engines and producing one combined confidence score.

You define a `calculate_confidence_score` function that takes the three user-facing scores (`face_score`, `speech_score`, `voice_score`) and produces a weighted average. A clean MVP starting point is face 40%, speech 35%, and voice 25%. If any score is unavailable (off or errored), its weight is redistributed proportionally among the active scores so the total always adds up correctly. An exponential moving average smooths the output so the displayed score does not jump erratically between updates.

ExamGuard comparison: ExamGuard also combined multiple signals into one decision score. The difference here is that 4 engines produce 3 user-facing scores, and they update at different rates.

3. State Management

What: Keeping the React UI in sync with data arriving from 4 engines at different rates.

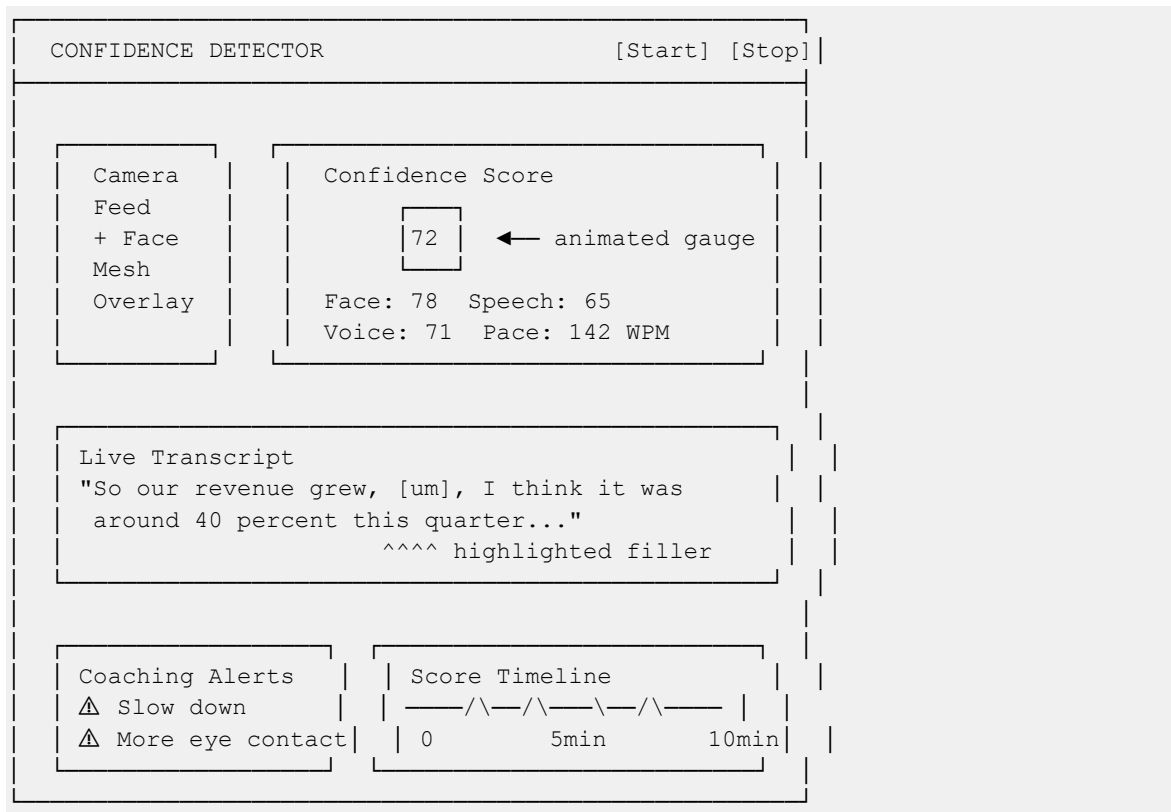
You define a single React state object that holds the status and latest data for each engine (face, speech, voice) along with a combined section for the overall score, score history timeline, and active coaching alerts. Each engine sub-object tracks whether it is running, its current score, its specific metrics (expression, transcript, pitch, etc.), and its last update timestamp. The key challenge is that face updates 15 times per second, so high-frequency data should use `useRef` instead of state to avoid freezing the UI with excessive re-renders.

Key challenge: Face updates 15x/sec. If every update triggers a React re-render, the UI freezes. Solution: batch updates, use `useRef` for high-frequency data, only re-render the score display at 4 FPS.

Confidence Detector connection: Bad state management = frozen UI = unusable product. This is a make-or-break skill.

4. Real-Time Dashboard Components

What: Building the visual components the user actually sees.



Components to build:

- Confidence gauge (animated circle or meter, 0-100)
- Live transcript with filler word highlighting
- Score timeline chart (line chart updating in real time)
- Sub-score bars (face, speech, voice)
- Coaching alert panel (fade in/out)
- Audio waveform display
- Session timer

Confidence Detector connection: This is the user-facing product. Everything else is invisible backend work. The dashboard IS the Confidence Detector.

5. Coaching Alert System

What: Generating real-time advice based on current metrics.

You define an array of coaching rules, each with an id, a condition function that checks current metrics (pace > 180, eye contact < 30%, filler ratio > 10%, volume too low, not smiling), a user-friendly message, a cooldown timer (30-120 seconds to prevent spamming), and a priority level. A `checkAlerts` function evaluates all rules against current metrics, fires only those whose cooldown has expired, sorts by priority, and returns at most 2 alerts at a time so the user is not overwhelmed.

Confidence Detector connection: Coaching alerts are what make this a TRAINING tool, not just a measurement tool. Without alerts, the user has to interpret the score themselves. With alerts, they get actionable advice in the moment.

6. Session Recording

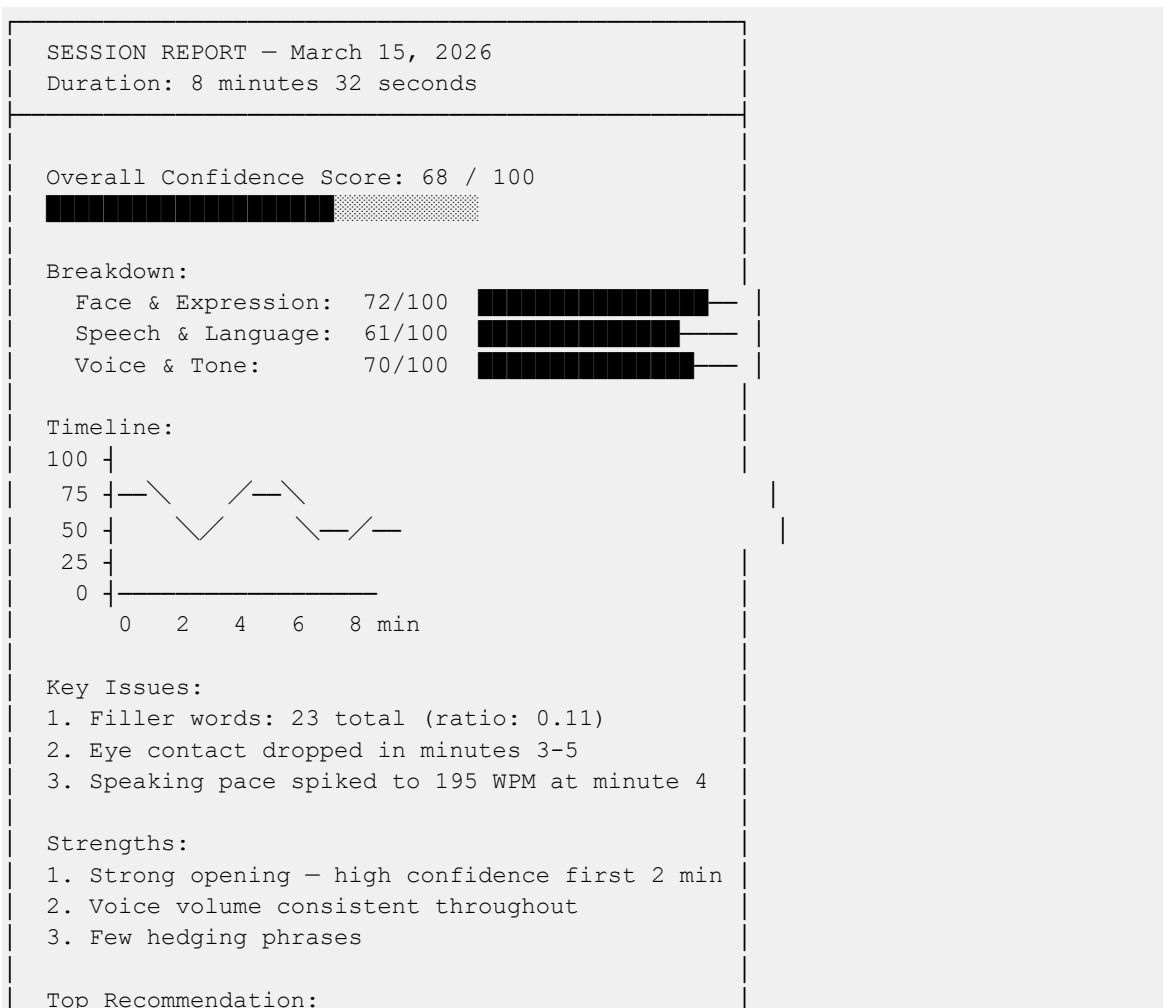
What: Storing metrics at regular intervals for post-session analysis.

You build a `SessionRecorder` class that samples all scores and metrics every 1 second using `setInterval`, storing each data point with a relative timestamp, all sub-scores (combined, face, speech, voice), pace, and filler count. When the session ends, the timer is cleared and a report is generated from the accumulated time-series data. This allows the post-session report to show how confidence changed over the duration of the presentation.

Confidence Detector connection: Without session recording, the user only sees the current moment. With it, they can see "I started at 80/100, dropped to 45/100 at minute 3, then recovered to 70/100." This is the data that enables meaningful improvement.

7. Report Generation

What: After the session ends, aggregate all data into a readable summary.



```
Focus on reducing filler words. Practice
pausing silently instead of saying "um."
```

Confidence Detector connection: The report is what the user takes away. It is the tangible output they can review, share with a coach, and compare against future sessions.

8. AI API Integration (Optional v2)

What: Sending snapshot data to a backend route that calls the Claude API for deep, context-aware feedback.

You send a POST request from the browser to **your own backend/proxy**, not directly to Anthropic with a secret API key in frontend code. The backend holds the provider key, forwards the session data (overall score, duration, filler count/ratio, hedge count, average pace, eye contact ratio, dominant expression, and a transcript excerpt), and returns structured feedback. This keeps credentials out of the browser and lets you add logging, rate limits, and fallback behavior later.

Confidence Detector connection: This is the "wow factor" for v1.0. Rule-based scoring is good. AI-powered feedback that says "In your third sentence, when you said 'I think maybe we could possibly consider...' — that quadruple hedge killed your authority. Try: 'We should consider...'" is transformative.

9. Performance Optimization

What: Keeping the app running smoothly at 15+ FPS with all engines active.

Performance Budget:

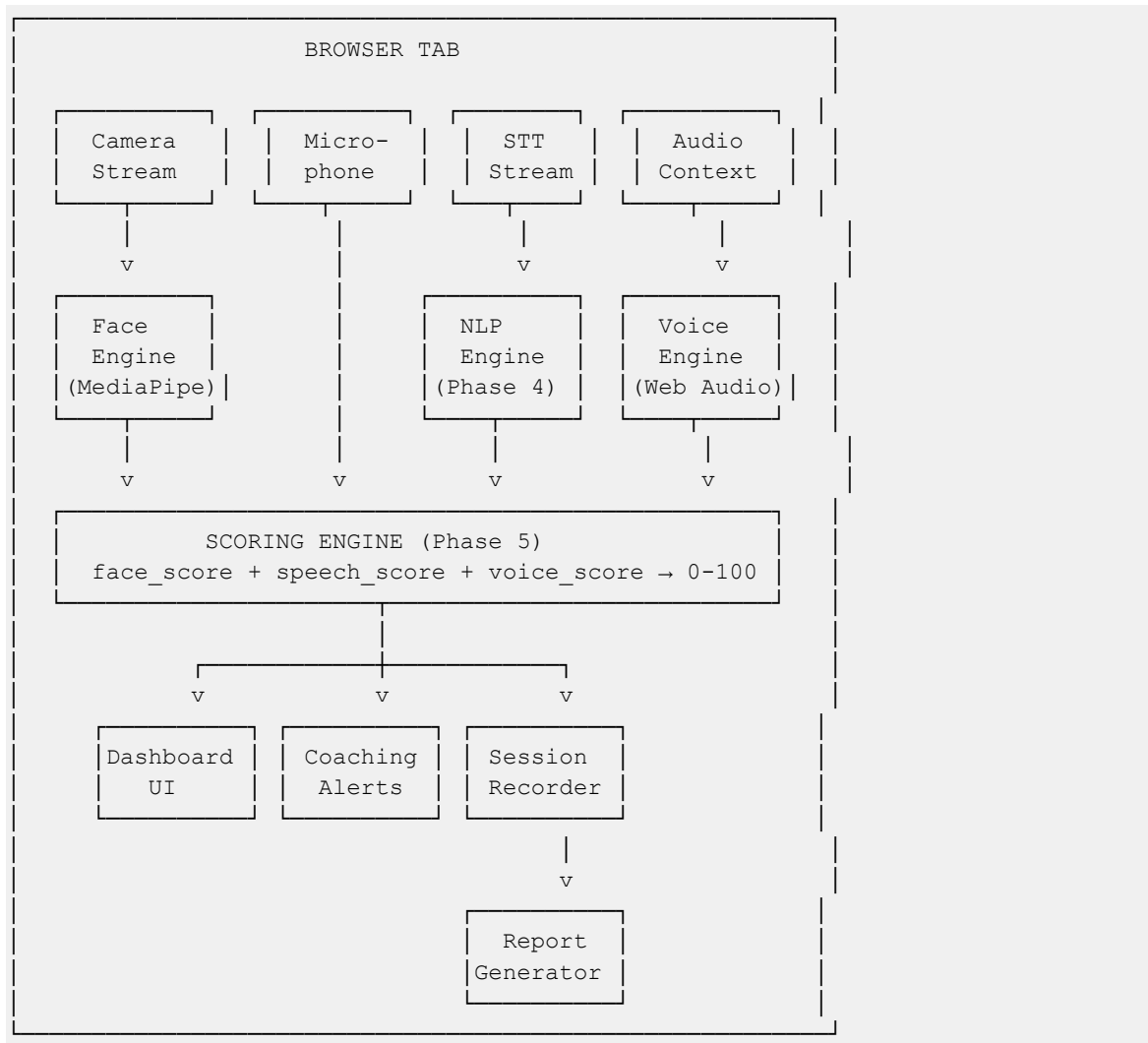
Component	Target Time
Face Mesh processing	< 30ms per frame
STT processing	0ms (browser handles it)
NLP analysis	< 5ms per sentence
Voice analysis	< 10ms per chunk
Score calculation	< 1ms
UI render	< 16ms (60 FPS budget)
Total per frame	< 62ms = ~16 FPS ✓

Optimization techniques:

- Frame skipping:** Process face every 2nd or 3rd camera frame
- Web Workers:** Offload NLP and voice analysis to background threads
- Throttled renders:** Update the score display at 4 FPS, not 15 FPS
- useRef for high-frequency data:** Store rapidly changing values in refs, not state
- Lazy analysis:** Run NLP only when a new sentence completes, not on every word
- Canvas over DOM:** Use canvas for waveforms and charts instead of DOM elements

ExamGuard comparison: ExamGuard had the same challenge with real-time YOLO processing. The optimization mindset is identical: skip frames, batch updates, profile bottlenecks. But with 4 engines instead of 1, the budget is tighter.

Integration Architecture



Component Architecture: How Files Are Organized

```
backend/
  main.py          - FastAPI server (routes for live + upload)
  face_engine.py   - blendshapes + pose detection
  speech_engine.py - Vosk STT + NLP analysis

frontend/
  src/App.jsx      - React app (home, live mode, upload mode, results)
  src/App.css      - Dark theme styling
```

WHY this split: The backend handles heavy ML work (MediaPipe for face mesh, Vosk for speech-to-text). The frontend handles UI and display only. The browser sends camera frames or audio to the backend, the backend runs the models and returns scores.

This is the same pattern as ExamGuard: Python backend does the AI processing, React frontend shows the results. If you understood ExamGuard's architecture, this will feel familiar. The only difference is more engines (face + speech + voice instead of just YOLO + MediaPipe).

Why not run everything in the browser? You could run MediaPipe and Web Speech API entirely in the browser (and the MVP versions do exactly that). But Vosk requires a backend, and putting all ML processing server-side gives you consistent performance across devices. The backend approach also makes it easier to add Claude API integration later without exposing API keys in frontend code.

Error Handling: What Happens When Things Break

Real-time systems break in predictable ways. Plan for each failure instead of discovering them during a demo.

Failure	What User Sees	System Response
Camera denied	"Camera access needed" message	Show step-by-step instructions to enable camera in browser settings
STT timeout (60s)	Nothing visible	Auto-restart recognition in the <code>onend</code> handler. User never notices
Face not detected	"No face detected" label	Score from voice + speech only. Redistribute face weight (0.40) proportionally to speech and voice
No audio in video	"No audio found" note	Score from face only. Show "Face Only" label so user knows why speech/voice scores are missing
Backend not running	"Cannot connect" alert	Clear message: "Run <code>python main.py</code> first." Do not show a cryptic network error
Slow device (<15 FPS)	Choppy video	Reduce <code>process_every</code> from 2 to 4 (skip more frames). Log FPS to console so you can diagnose

The principle: Never crash silently. Never show a raw error. Always tell the user what happened and what to do about it. If one engine fails, the others keep running and the scoring formula adjusts automatically.

Performance Budget: Real Numbers

At 30 FPS, you have 33ms per frame. Here is where that time goes:

Component	Time	When
-----------	------	------

MediaPipe face detect	~15ms	Every 2nd frame
MediaPipe pose detect	~10ms	Every 2nd frame
Blendshape scoring	<1ms	Every 2nd frame
Speech-to-text (Vosk)	~5ms per chunk	Continuous
NLP analysis	<1ms	Per sentence
Video encoding (MJPEG)	~3ms	Every frame
Total per processed frame	~34ms	Target: under 40ms

WHY we process every 2nd frame: 34ms per processed frame is already over the 33ms budget for 30 FPS. If you process every frame, you will drop frames and the video will stutter. Processing every 2nd frame means each frame costs ~17ms on average (34ms processing spread across 2 frames). That gives a comfortable margin.

What if 17ms is still too slow? Switch to every 3rd frame. You lose some responsiveness (face expression updates 10x/sec instead of 15x/sec) but the user will not notice. Below 8x/sec the delay becomes visible.

How to measure this yourself: Open Chrome DevTools > Performance tab > Record a 10-second session > Look at the "Main" thread. If you see long yellow bars (JS execution) exceeding 33ms, you are dropping frames.

Resources

Multi-Engine / Orchestration

- [Web Workers MDN Guide](https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API/Using_web_workers) — Running code in background threads
- [React + Web Workers](https://blog.logrocket.com/web-workers-react/) — Integrating workers with React
- [requestAnimationFrame](https://developer.mozilla.org/en-US/docs/Web/API/window/requestAnimationFrame) — Efficient animation loop

Dashboard & Visualization

- [Chart.js](https://www.chartjs.org/) — Simple, flexible chart library (good for score timeline)
- [Recharts](https://recharts.org/) — React-specific charting library
- [React Gauge Chart](https://www.npmjs.com/package/react-gauge-chart) — Gauge/meter component
- [Canvas API](https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API) — For custom waveform rendering

React State Management

- [React useRef vs useState](https://react.dev/reference/react/useRef) — When to use which (critical for real-time apps)

- [React Performance Optimization](https://react.dev/reference/react/memo) — `React.memo`, `useMemo`, `useCallback`

Performance

- [Chrome DevTools Performance Tab](https://developer.chrome.com/docs/devtools/performance/) — Profiling your app
- [Web Performance Patterns](https://web.dev/performance/) — General web performance guide

AI API Integration

- [Anthropic API Docs](https://docs.anthropic.com/) — Claude API reference
- [Anthropic SDK for JS](https://www.npmjs.com/package/@anthropic-ai/sdk) — Official JavaScript SDK

Similar Products (Inspiration)

- [Yoodli](https://yoodli.ai/) — AI speech coach (commercial product doing similar things)
- [Poised](https://www.poised.com/) — Real-time communication coach
- [Orai](https://www.orai.com/) — Public speaking practice app